

```

1  public class NodeTest {
2
3      public static void main (String[] args)
4      {
5          int [] arr = {3, 2, 4, 5, 3, 1};
6          Node<Integer> lst1 = build(arr);
7          show(lst1);
8          show(removeAllEvenDigits(lst1));
9          show(sumAndAddDigits(lst1));
10
11     }
12
13
14     public static Node<Integer> sumAndAddDigits(Node<Integer> lst) {
15         Node<Integer> prevInsert = null;
16         Node<Integer> pos = lst;
17
18         while (pos != null) {
19             lst = insert(lst, prevInsert, pos.getValue() + pos.getNext().getValue());
20
21             if (prevInsert == null) {
22                 prevInsert = lst.getNext().getNext();
23             } else {
24                 prevInsert = prevInsert.getNext().getNext().getNext();
25             }
26             pos = pos.getNext().getNext();
27         }
28
29         return lst;
30     }
31
32     public static Node<Integer> removeAllEvenDigits(Node<Integer> lst){
33         Node<Integer> pos = lst;
34         while (pos != null){
35             if (pos.getValue() %2 == 0)
36                 lst = remove(lst, pos);
37             pos = pos.getNext();
38         }
39         return lst;
40     }
41
42
43     //--- בניית שרשרת מההתחלה לסוף ---
44     public static Node<Integer> build (int [] arr)
45     {
46         Node<Integer> lst = new
47             Node<Integer>(arr[0]);
48         Node<Integer> pos = lst;
49

```

```
50     for (int i = 1 ; i < arr.length ; i++)
51     {
52         pos.setNext (new Node<Integer> (arr[i]));
53         pos = pos.getNext();
54     }
55     return lst;
56 }
57
58 // prints list
59 public static void show(Node<Integer> lst)
60 {
61     System.out.print("[");
62     while (lst != null)
63     {
64         System.out.print(lst.toString());
65         if (lst.hasNext())
66             System.out.print(", ");
67         lst = lst.getNext();
68     }
69     System.out.println("]");
70 }
71
72 // returns the sum of the list
73 public static int sumList(Node<Integer> lst){
74     int sum = 0;
75     while (lst != null)
76     {
77         sum += lst.getValue();
78         lst = lst.getNext();
79     }
80
81     return sum;
82 }
83
84 // build a list with num amount of nodes
85 public static Node<Integer> buildToNum (int num)
86 {
87     Node<Integer> lst = new Node<Integer>(0);
88     Node<Integer> pos = lst;
89
90     for (int i = 1 ; i < num ; i++)
91     {
92         pos.setNext (new Node<Integer> (i));
93         pos = pos.getNext();
94     }
95     return lst;
96 }
97
98 // checks if a given num is in the list
```

```

99  public static boolean isNumberInList(int num, Node<Integer> lst){
100      while (lst != null)
101      {
102          if (lst.getValue() == num)
103              return true;
104          lst = lst.getNext();
105      }
106      return false;
107  }
108
109  // checks if a given num is in the list but with recursion
110  public static boolean isNumberInListReq(int num, Node<Integer> lst){
111      if (lst == null) return false;
112      if (lst.getValue() == num) return true;
113
114      return isNumberInListReq(num, lst.getNext());
115  }
116
117  // checks if the list is sorted
118  public static boolean isSorted(Node<Integer> lst){
119      while (lst.getNext() != null)
120      {
121          if (lst.getValue() > lst.getNext().getValue())
122              return false;
123
124          lst = lst.getNext();
125      }
126      return true;
127  }
128
129  // checks if the array is sorted with recursion
130  public static boolean isSortedReq(Node<Integer> lst){
131      if (lst.getNext() == null) return true;
132      return (lst.getValue() < lst.getNext().getValue() && isSortedReq(lst.
getNext()));
133  }
134
135
136  // removes a value in the list
137  public static <T> Node<T> remove(Node<T> lst, Node<T> pos)
138  {
139      if(pos == lst) // if the pos == the first node in list
140          return lst.getNext();
141
142      Node<T> prev = lst;
143
144      while(prev.getNext() != pos)
145          prev = prev.getNext();
146

```

```

147     prev.setNext(pos.getNext());
148     return lst;
149 }
150
151 //adds a new value in list
152 public static <T> Node<T> insert(Node<T> lst, Node<T> pos , T x)
153 {
154     if(pos == null)
155         lst = new Node<T>(x,lst);
156     else
157     {
158         Node<T> temp = new Node<T>(x, pos.getNext());
159         pos.setNext(temp);
160     }
161     return lst;
162 }
163
164 // list length
165 public static <T> int numOfItemsInList(Node<T> first)
166 {
167     int count=0;
168     Node<T> p = first;
169     while (p != null)
170     {
171         count++;
172         p = p.getNext();
173     }
174     return count;
175 }
176
177 }
178 /*
179 פלט
180 [ 3, 2, 4, 5, 3, 1]
181 [ 3, 5, 3, 1]
182 [ 8, 3, 5, 4, 3, 1]
183 */
184

```